1. Program for Stack Implementation in C

```c
#include <stdio.h>
int MAXSIZE = 8;
int stack[8];
int top = -1;

/* Check if the stack is empty */
int isempty(){
   if(top == -1)
      return 1;
   else
      return 0;
}

/* Check if the stack is full */
int isfull(){
   if(top == MAXSIZE)
      return 1;
   else
      return 0;
}

/* Function to return the topmost element in the stack */
int peek(){
   return stack[top];
}

/* Function to delete from the stack */
int pop(){
   int data;
   if(!isempty()) {
      data = stack[top];
      top = top - 1;
      return data;
   } else {
      printf("Could not retrieve data, Stack is empty.\n");
   }
}

/* Function to insert into the stack */
int push(int data){
   if(!isfull()) {
      top = top + 1;
```

```
      stack[top] = data;
   } else {
      printf("Could not insert data, Stack is full.\n");
   }
}

// Display stack data
int display()
{
   while(!isempty()) {
    int data = pop();
    printf("%d\n",data);
    }
}

/* Main function */
int main(){
   push(44);
   push(10);
   push(62);
   push(123);
   push(15);
   printf("Element at top of the stack: %d\n" ,peek());
   printf("Elements: \n");
// Print stack data
   display();
   printf("Stack is full: %s\n",isfull()?"true":"false");
   printf("Stack is empty: %s\n",isempty()?"true":"false");

   return 0;
}
```

Output:-

Element at top of the stack: 15

Elements:

15

123

62

10

44

Stack full: false

Stack empty: true

2. Program for Queue Implementation in C

```
#include <stdio.h>
#define MAX_SIZE 10
int queue[MAX_SIZE],front=-1,rear=-1;
int isFull()
{
    return rear==MAX_SIZE-1;
}
int isEmpty()
{
    return front==-1||front>rear;
}
void enqueue(int element)
{
    if(isFull())
    {
        printf("Queue overflow\n");
        return;
    }
    if(front==-1)
    {
        front=0;
    }
    rear++;
    queue[rear]=element;
    printf("Element %d inserted\n",element);
}
int dequeue()
{
    if(isEmpty())
    {
        printf("Queue underflow\n");
        return -1;
    }
    int element=queue[front];
```

```
    front++;
    return element;
}
void display()
{
    if(isEmpty())
    {
        printf("Queue is empty\n");
        return;
    }
    printf("Queue elements: ");
    int i=front;
    while(i!=rear)
    {
        printf("%d ",queue[i]);
        i++;
    }
    printf("%d\n",queue[rear]);
}
int main()
{
    enqueue(10);
    enqueue(100);
    enqueue(1000);
    display();
    printf("Dequeued element: %d\n",dequeue());
    display();
    return 0;
}
```

Output:-

Element 10 inserted
Element 100 inserted
Element 1000 inserted
Queue elements: 10 100 1000
Dequeued element: 10
Queue elements: 100 1000

3. Program for Circular Queue Implementation in C

```
// C Program to implement the circular queue in c using arrays
#include <stdio.h>
```

```c
// Define the maximum size of the queue
#define MAX_SIZE 5

    // Declare the queue array and front, rear variables
    int queue[MAX_SIZE];
int front = -1, rear = -1;

// Function to check if the queue is full
int isFull()
{
    // If the next position is the front, the queue is full
    return (rear + 1) % MAX_SIZE == front;
}

// Function to check if the queue is empty
int isEmpty()
{
    // If the front hasn't been set, the queue is empty
    return front == -1;
}

// Function to enqueue (insert) an element
void enqueue(int data)
{
    // If the queue is full, print an error message and
    // return
    if (isFull()) {
        printf("Queue overflow\n");
        return;
    }
    // If the queue is empty, set the front to the first
    // position
    if (front == -1) {
        front = 0;
    }
    // Add the data to the queue and move the rear pointer
    rear = (rear + 1) % MAX_SIZE;
    queue[rear] = data;
    printf("Element %d inserted\n", data);
}

// Function to dequeue (remove) an element
int dequeue()
{
```

```
    // If the queue is empty, print an error message and
    // return -1
    if (isEmpty()) {
        printf("Queue underflow\n");
        return -1;
    }
    // Get the data from the front of the queue
    int data = queue[front];
    // If the front and rear pointers are at the same
    // position, reset them
    if (front == rear) {
        front = rear = -1;
    }
    else {
        // Otherwise, move the front pointer to the next
        // position
        front = (front + 1) % MAX_SIZE;
    }
    // Return the dequeued data
    return data;
}

// Function to display the queue elements
void display()
{
    // If the queue is empty, print a message and return
    if (isEmpty()) {
        printf("Queue is empty\n");
        return;
    }
    // Print the elements in the queue
    printf("Queue elements: ");
    int i = front;
    while (i != rear) {
        printf("%d ", queue[i]);
        i = (i + 1) % MAX_SIZE;
    }
    // Print the last element
    printf("%d\n", queue[rear]);
}

// Main function
int main()
{
    // Enqueue some elements
```

```
    enqueue(10);
    enqueue(20);
    enqueue(30);
    // Display the queue
    display();

    // Dequeue an element and print it
    printf("Dequeued element: %d\n", dequeue());
    // Display the queue again
    display();

    // End of main function
    return 0;
}
```

Output:-

Element 10 inserted

Element 20 inserted

Element 30 inserted

Queue elements: 10 20 30

Dequeued element: 10

Queue elements: 20 30

4. Program for First Come First Serve (FCFS) Implementation in C

```
                //Program of First Come First Serve (FCFS)
#include <stdio.h>
int main()
{
    int b[5], i, w[5], c[5],t[5];
    float sum1, avg1, sum2, avg2, sum3, avg3;
    printf("Enter value of various processes: \n");
    for (i = 0; i <= 4; i++)
    {
        printf("P[%d]: ", i);
        scanf("%d", &b[i]);
    }
```

```
    printf("\nBurst Time is: \n");
    for (i = 0; i <= 4; i++)
    {
        printf("P[%d]: ", i);
        printf("%d\n", b[i]);
    }
    printf("\nWaiting time is: \n");
    for (i = 0; i <= 4; i++)
    {
        w[i] = 0;
        w[i] = b[i - 1] + w[i - 1];
        printf("%d\t", w[i]);
    }
    printf("\n\n");
    sum1 = w[0] + w[1] + w[2] + w[3] + w[4];
    avg1 = sum1 / 5;
    printf("Average waiting time is %.2f", avg1);
    printf("\n\nCompletion time: \n");
    for (i = 0; i <= 4; i++)
    {
        c[i] = c[i - 1] + b[i];       //c[i] = w[i] + b[i];
        printf("%d\t", c[i]);
    }
    printf("\n\n");
    sum2 = c[0] + c[1] + c[2] + c[3] + c[4];
    avg2 = sum2 / 5;
    printf("Average completion time is %.2f", avg2);
    /*printf("\n\nTurn around time: \n");
    for (i = 0; i <= 4; i++)
    {
        t[i] = w[i] + b[i];
        printf("%d\t", t[i]);
    }
    printf("\n\n");
    sum3 = t[0] + t[1] + t[2] + t[3] + t[4];
    avg3 = sum3 / 5;
    printf("Average turnaround time is %.2f", avg3);*/
return 0;
}
```

Output:-

Enter value of various processes:

P[0]: 2

P[1]: 4

P[2]: 3

P[3]: 1

P[4]: 5

Burst Time is:

P[0]: 2

P[1]: 4

P[2]: 3

P[3]: 1

P[4]: 5

Waiting time is:

0 2 6 9 10

Average waiting time is 5.40

Completion time:

2 6 9 10 15

Average completion time is 8.40

5. Program for Shortest Time First with Non-Preemption (SJF) Implementation in C

```
//Program of SJF with Non-Preemption.
```

```c
#include <stdio.h>
int main()
{
    int i,n,temp;
    float sum1,avg1,sum2,avg2,sum3,avg3;
    printf("Enter number of processes: ");
    scanf("%d",&n);
    int b[n],w[n],c[n],t[n];
    printf("Enter value of various processes: \n");
    for(i=0;i<n;i++)
    {
        printf("P[%d]: ",i);
        scanf("%d",&b[i]);
    }
    for(int a=0;a<n-1;a++)
    {
        for(int c=0;c<n-a-1;c++)
        {
            if(b[c]>b[c+1])
            {
                temp=b[c];
                b[c]=b[c+1];
                b[c+1]=temp;
            }
        }
    }
    printf("If consider all processes arrival time is 0.\nThen Gantt Chart: \n");
    for(i=0;i<n;i++)
    {
        printf("%d\t",b[i]);
    }
    printf("\nWaiting time: \n");
    for(i=0;i<n;i++)
    {
        w[i]=0;
        w[i]=w[i-1]+b[i-1];
        printf("%d\t",w[i]);
    }
    printf("\n");
    sum1 = w[0] + w[1] + w[2] + w[3] + w[4];
    avg1 = sum1 / n;
    printf("Average waiting time is %.2f\n", avg1);
    printf("Completion time: \n");
    for(i=0;i<n;i++)
    {
```

```
        c[i] = c[i-1] + b[i];
        printf("%d\t", c[i]);
    }
    printf("\n");
    sum2 = c[0] + c[1] + c[2] + c[3] + c[4];
    avg2 = sum2 / n;
    printf("Average completion time is %.2f\n", avg2);
    return 0;
}
```

Output:-

Enter number of processes: 5

Enter value of various processes:

P[0]: 3

P[1]: 8

P[2]: 5

P[3]: 4

P[4]: 6

If consider all processes arrival time is 0.

Then Gantt Chart:

3 4 5 6 8

Waiting time:

0 3 7 12 18

Average waiting time is 8.00

Completion time:

3 7 12 18 26

Average completion time is 13.20

6. Program for Round_Robin Implementation in C

```c
#include <stdio.h>

#define MAX_PROCESSES 5

int main() {
    int burst_time[MAX_PROCESSES], waiting_time[MAX_PROCESSES],
turnaround_time[MAX_PROCESSES];
    int i, j, n = MAX_PROCESSES, time_quantum, total_time = 0;
    int remaining_time[MAX_PROCESSES];

    // Input burst times
    printf("Enter burst times for %d processes:\n", n);
    for (i = 0; i < n; i++) {
        printf("Process %d: ", i + 1);
        scanf("%d", &burst_time[i]);
        remaining_time[i] = burst_time[i]; // Initialize remaining time
    }

    // Input time quantum
    printf("Enter time quantum: ");
    scanf("%d", &time_quantum);

    // Round Robin Scheduling
    while (1) {
        int done = 1; // Check if all processes are done
        for (i = 0; i < n; i++) {
            if (remaining_time[i] > 0) {
                done = 0; // There is at least one process not done
                if (remaining_time[i] > time_quantum) {
                    total_time += time_quantum;
                    remaining_time[i] -= time_quantum;
                } else {
                    total_time += remaining_time[i];
                    waiting_time[i] = total_time - burst_time[i];
                    remaining_time[i] = 0;
                }
            }
        }
        if (done) break; // Exit if all processes are done
    }

    // Calculate turnaround time
    for (i = 0; i < n; i++) {
        turnaround_time[i] = burst_time[i] + waiting_time[i];
    }
```

```
    // Output results
    printf("\nProcess\tBurst Time\tWaiting Time\tTurnaround Time\n");
    for (i = 0; i < n; i++) {
        printf("%d\t%d\t\t%d\t\t%d\n", i + 1, burst_time[i], waiting_time[i],
turnaround_time[i]);
    }

    return 0;
}
```

Output:-

Enter burst times for 5 processes:

Process 1: 8

Process 2: 7

Process 3: 2

Process 4: 5

Process 5: 4

Enter time quantum: 2

Process	Burst Time	Waiting Time	Turnaround Time
1	8	17	25
2	7	19	26
3	2	4	6
4	5	18	23
5	4	14	18

7. Program for Priority_Scheduling Implementation in C

```c
#include <stdio.h>
int main(){
    int i,j,n,temp;
    float sum1=0,sum2=0,avg1,avg2;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int b[n],w[n],c[n],p[n];
    printf("Enter value of various burst time processes: \n");
    for(i=0;i<n;i++){
        printf("\nEnter P[%d]: ",i+1);
        scanf("%d",&b[i]);
    }
    printf("Enter value of various priority processes: \n");
    for(i=0;i<n;i++){
        printf("\nEnter P[%d]: ",i+1);
        scanf("%d",&p[i]);
    }
    for(i=0;i<n-1;i++){
        for(j=i+1;j<n;j++){
            if(p[i]>p[j]){
                // Swap burst_time
                temp=b[i];
                b[i]=b[j];
                b[j]=temp;
                // Swap priority
                temp=p[i];
                p[i]=p[j];
                p[j]=temp;
            }
        }
    }
    for(i=0;i<n;i++){
        w[i]=0;
        w[i]=b[i-1]+w[i-1];
    }
    for(i=0;i<n;i++){
        c[i]=w[i]+b[i];
    }
    printf("\nProcess\tBurst Time\tPriority\tWaiting Time\tCompletion Time\n");
    for(i=0;i<n;i++){
        printf("\nP[%d]\t\t%d\t\t%d\t\t%d\t\t%d\n",i+1,b[i],p[i],w[i],c[i]);
```

```
    }
    for(i=0;i<n;i++){
        sum1+=w[i];
        sum2+=c[i];
    }
    avg1=sum1/n;
    avg2=sum2/n;
    printf("\nAverage Waiting Time: %.2f\n",avg1);
    printf("\nAverage Completion Time: %.2f\n",avg2);
    return 0;
}
```

Output:-

Enter number of processes: 5

Enter value of various burst time processes:

Enter P[1]: 8

Enter P[2]: 7

Enter P[3]: 5

Enter P[4]: 3

Enter P[5]: 4

Enter value of various priority processes:

Enter P[1]: 8

Enter P[2]: 9

Enter P[3]: 4

Enter P[4]: 5

Enter P[5]: 7

Process	Burst Time	Priority	Waiting Time	Completion Time
P[1]	5	4	0	5
P[2]	3	5	5	8
P[3]	4	7	8	12
P[4]	8	8	12	20
P[5]	7	9	20	27

Average Waiting Time: 9.00

Average Completion Time: 14.40

8. Program for Banker’s Algorithm Implementation in C

```c
#include <stdio.h>
int main(){
    int n,m,i,j,k;
    n=5,m=3;
    /*int allocation[n][m],max[n][m],available[m];
    printf("Enter number of processes: ");
    scanf("%d",&n);
    printf("Enter number of resourses: ");
    scanf("%d",&m);
    printf("Enter allocation of processes: \n");
    for(i=0;i<n;i++){
        for(j=0;j<m;j++){
            printf("Enter value of [%d][%d]: ",i+1,j+1);
            scanf("%d",&allocation[i][j]);
        }
    }
    printf("Enter maximum need of processes: \n");
    for(i=0;i<n;i++){
        for(j=0;j<m;j++){
            printf("Enter value of [%d][%d]: ",i+1,j+1);
            scanf("%d",&max[i][j]);
        }
    }
    printf("Enter available resouuces of processes: \n");
    for(j=0;j<m;j++){
        printf("Enter value of [%d]: ",j+1);
            scanf("%d",&available[j]);
    }*/
    int allocation[5][3]={
        {0,1,0}, {2,0,0}, {3,0,2}, {2,1,1}, {0,0,2}
    };
    int max[5][3]={
        {7,5,3}, {3,2,2}, {9,0,2}, {2,2,2}, {4,3,3}
    };
    int available[3]={3,3,2};
    int f[n],ans[n],ind=0,y=0;
    for(k=0;k<n;k++){
        f[k]=0;
    }
    int need[n][m];
    for(i=0;i<n;i++){
        for(j=0;j<n;j++){
```

```
            need[i][j]=max[i][j]-allocation[i][j];
        }
    }
    for(k=0;k<n;k++){
        for(i=0;i<n;i++){
            if(f[i]==0){
                int flag=0;
                for(j=0;j<m;j++){
                    if(need[i][j]>available[j]){
                        flag=1;
                        break;
                    }
                }
                if(flag==0){
                    ans[ind++]=i;
                    for(y=0;y<m;y++){
                        available[y]+=allocation[i][y];
                    }
                    f[i]=1;
                }
            }
        }
    }
    int flag=1;
    for(i=0;i<n;i++){
        if(f[i]==0){
            flag=0;
            printf("The following sequence is not a safe.");
            break;
        }
    }
    if(flag==1){
        printf("The following sequence is a safe.\n");
        for(i=0;i<n-1;i++){
            printf(" P%d ->",ans[i]);
        }
        printf(" P%d",ans[n-1]);
    }
    return 0;
}
```

Output:-

The following sequence is a safe.

P1 -> P3 -> P4 -> P0 -> P2

9. Program for Quick Sort Implementation in C

```c
// Quick sort in C

#include <stdio.h>

// function to swap elements
void swap(int *a, int *b) {
  int t = *a;
  *a = *b;
  *b = t;
}

// function to find the partition position
int partition(int array[], int low, int high) {

  // select the rightmost element as pivot
  int pivot = array[high];

  // pointer for greater element
  int i = (low - 1);

  // traverse each element of the array
  // compare them with the pivot
  for (int j = low; j < high; j++) {
    if (array[j] <= pivot) {

      // if element smaller than pivot is found
      // swap it with the greater element pointed by i
      i++;

      // swap element at i with element at j
      swap(&array[i], &array[j]);
    }
  }

  // swap the pivot element with the greater element at i
  swap(&array[i + 1], &array[high]);

  // return the partition point
  return (i + 1);
}

void quickSort(int array[], int low, int high) {
```

```
  if (low < high) {

    // find the pivot element such that
    // elements smaller than pivot are on left of pivot
    // elements greater than pivot are on right of pivot
    int pi = partition(array, low, high);

    // recursive call on the left of pivot
    quickSort(array, low, pi - 1);

    // recursive call on the right of pivot
    quickSort(array, pi + 1, high);
  }
}

// function to print array elements
void printArray(int array[], int size) {
  for (int i = 0; i < size; ++i) {
    printf("%d  ", array[i]);
  }
  printf("\n");
}

// main function
int main() {
  int data[] = {8, 7, 2, 1, 0, 9, 6};

  int n = sizeof(data) / sizeof(data[0]);

  printf("Unsorted Array\n");
  printArray(data, n);

  // perform quicksort on data
  quickSort(data, 0, n - 1);

  printf("Sorted array in ascending order: \n");
  printArray(data, n);
  return 0;
}
```

Output:-

Unsorted Array

8 7 2 1 0 9 6

Sorted array in ascending order:

0 1 2 6 7 8 9

10. Program for Merge Sort Implementation in C

```c
// Merge sort in C

#include <stdio.h>

// Merge two subarrays L and M into arr
void merge(int arr[], int p, int q, int r) {

  // Create L ← A[p..q] and M ← A[q+1..r]
  int n1 = q - p + 1;
  int n2 = r - q;

  int L[n1], M[n2];

  for (int i = 0; i < n1; i++)
    L[i] = arr[p + i];
  for (int j = 0; j < n2; j++)
    M[j] = arr[q + 1 + j];

  // Maintain current index of sub-arrays and main array
  int i, j, k;
  i = 0;
  j = 0;
  k = p;

  // Until we reach either end of either L or M, pick larger among
  // elements L and M and place them in the correct position at A[p..r]
  while (i < n1 && j < n2) {
    if (L[i] <= M[j]) {
      arr[k] = L[i];
      i++;
    } else {
      arr[k] = M[j];
      j++;
    }
    k++;
  }
```

```c
  // When we run out of elements in either L or M,
  // pick up the remaining elements and put in A[p..r]
  while (i < n1) {
    arr[k] = L[i];
    i++;
    k++;
  }

  while (j < n2) {
    arr[k] = M[j];
    j++;
    k++;
  }
}

// Divide the array into two subarrays, sort them and merge them
void mergeSort(int arr[], int l, int r) {
  if (l < r) {

    // m is the point where the array is divided into two subarrays
    int m = l + (r - l) / 2;

    mergeSort(arr, l, m);
    mergeSort(arr, m + 1, r);

    // Merge the sorted subarrays
    merge(arr, l, m, r);
  }
}

// Print the array
void printArray(int arr[], int size) {
  for (int i = 0; i < size; i++)
    printf("%d ", arr[i]);
  printf("\n");
}

// Driver program
int main() {
  int arr[] = {6, 5, 12, 10, 9, 1};
  int size = sizeof(arr) / sizeof(arr[0]);

  mergeSort(arr, 0, size - 1);

  printf("Sorted array: \n");
```

```
  printArray(arr, size);
  return 0;
}
```

Output:-

Sorted array:

1 5 6 9 10 12

11. Program for Page Replacement FIFO Implementation in C

```
#include <stdio.h>
int main()
{
    int frames, pages[50], temp[50];
    int n, i, j, k, pagefault = 0, index = 0, found;
    printf("Enter the number of pages: ");
    scanf("%d", &n);
    printf("Enter the number of frames: ");
    scanf("%d", &frames);
    printf("Enter %d pages data: \n",n);
    for (i = 0; i < n; i++)
    {
        printf("Pages[%d]: ",i+1);
        scanf("%d", &pages[i + 1]);
    }
    for (i = 0; i < frames; i++)
    {
        temp[i + 1] = -1;
    }
    for (i = 0; i < n; i++)
    {
        found = 0;
        for (j = 0; j < frames; j++)
        {
            if (pages[i + 1] == temp[j + 1])
            {
                found = 1;
                break;
```

```
            }
        }
        if (!found)
        {
            // PAGE REPLACEMENT FIFO
            temp[index] = pages[i + 1];
            index = (index + 1) % frames;
            pagefault++;
            printf("%d", &pages[i + 1]);
            for (k = 0; k < frames; k++)
            {
                if (temp[k] != -1)
                {
                    printf("%d ", temp[k]);
                }
                else
                {
                    printf("- ");
                }
            }
            printf("\tYes\n");
        }
        else
        {
            // Page hit
            printf("%d", &pages[i + 1]);
            for (k = 0; k < frames; k++)
            {
                if (temp[k] != -1)
                {
                    printf("%d ", temp[k]);
                }
                else
                {
                    printf("- ");
                }
            }
            printf("\tNo\n");
        }
    }
    printf("\nPage Faults: %d\n", pagefault);
    return 0;
}
```

Output:-

Enter the number of pages: 5

Enter the number of frames: 3

Enter 5 pages data:

Pages[1]: 5

Pages[2]: 8

Pages[3]: 7

Pages[4]: 3

Pages[5]: 2

5 - - Yes

5 8 - Yes

5 8 7 Yes

3 8 7 Yes

3 2 7 Yes

Page Faults: 5

12. Program for Page Replacement Optimal Implementation in C

```c
#include <stdio.h>

int main() {
    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2};
    int n = sizeof(pages) / sizeof(pages[0]);
    int frames[3] = {-1, -1, -1};
    int i, j, k, pageFaults = 0, found, index, farthest;

    printf("Page\tFrames\t\tPage Fault\n");

    for (i = 0; i < n; i++) {
        found = 0;
```

```
        // Check if page is already in frame
        for (j = 0; j < 3; j++) {
            if (frames[j] == pages[i]) {
                found = 1;
                break;
            }
        }

        if (found) {
            // Page found: No page fault
            printf("%d\t", pages[i]);
            for (j = 0; j < 3; j++) {
                if (frames[j] != -1)
                    printf("%d ", frames[j]);
                else
                    printf("- ");
            }
            printf("\tNo\n");
            continue;
        }

        // Page not found: Page fault
        pageFaults++;

        // Empty slot available
        for (j = 0; j < 3; j++) {
            if (frames[j] == -1) {
                frames[j] = pages[i];
                found = 1;
                break;
            }
        }

        // If no empty slot, apply optimal replacement
        if (!found) {
            int farthestIndex = -1;
            int pos = -1;
            for (j = 0; j < 3; j++) {
                int nextUse = -1;
                for (k = i + 1; k < n; k++) {
                    if (frames[j] == pages[k]) {
                        nextUse = k;
                        break;
                    }
```

```
                }

                if (nextUse == -1) {
                    pos = j;
                    break;
                } else if (nextUse > farthestIndex) {
                    farthestIndex = nextUse;
                    pos = j;
                }
            }
            frames[pos] = pages[i];
        }

        printf("%d\t", pages[i]);
        for (j = 0; j < 3; j++) {
            if (frames[j] != -1)
                printf("%d ", frames[j]);
            else
                printf("- ");
        }
        printf("\tYes\n");
    }

    printf("\nTotal Page Faults: %d\n", pageFaults);
    return 0;
}
```

Output:-

Page	Frames	Page Fault
7	7 - -	Yes
0	7 0 -	Yes
1	7 0 1	Yes
2	2 0 1	Yes
0	2 0 1	No
3	2 0 3	Yes
0	2 0 3	No

4 2 4 3 Yes

2 2 4 3 No

Total Page Faults: 6

13. Program for Page Replacement LRU (Least Recently Used) Implementation in C

```c
#include <stdio.h>

int main() {
    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2}; // Example reference string
    int n = sizeof(pages) / sizeof(pages[0]);

    int frames[3] = {-1, -1, -1};     // Frame array
    int time[3] = {0, 0, 0};          // To store last used time of each frame
    int i, j, pageFaults = 0, currentTime = 0;

    printf("Page\tFrames\t\tPage Fault\n");

    for (i = 0; i < n; i++) {
        int page = pages[i];
        int found = 0;

        // Check if page is already in frame
        for (j = 0; j < 3; j++) {
            if (frames[j] == page) {
                found = 1;
                time[j] = currentTime++;  // Update usage time
                break;
            }
        }

        if (found) {
            printf("%d\t", page);
            for (j = 0; j < 3; j++)
                (frames[j] != -1) ? printf("%d ", frames[j]) : printf("- ");
            printf("\tNo\n");
            continue;
        }

        // Page Fault
```

```
        pageFaults++;

        int pos = -1;
        for (j = 0; j < 3; j++) {
            if (frames[j] == -1) {
                pos = j;
                break;
            }
        }

        if (pos == -1) {
            // No empty frame, find LRU
            int lru = time[0];
            pos = 0;
            for (j = 1; j < 3; j++) {
                if (time[j] < lru) {
                    lru = time[j];
                    pos = j;
                }
            }
        }

        frames[pos] = page;
        time[pos] = currentTime++;

        printf("%d\t", page);
        for (j = 0; j < 3; j++)
            (frames[j] != -1) ? printf("%d ", frames[j]) : printf("- ");
        printf("\tYes\n");
    }

    printf("\nTotal Page Faults: %d\n", pageFaults);
    return 0;
}
```

Output:-

Page	Frames	Page Fault
7	7 - -	Yes
0	7 0 -	Yes

1	7 0 1	Yes
2	2 0 1	Yes
0	2 0 1	No
3	2 0 3	Yes
0	2 0 3	No
4	4 0 3	Yes
2	4 0 2	Yes

Total Page Faults: 7